

Technical Summary:

ReST-MCTS: LLM Self-Training via Process Reward Guided Tree Search

Kai-Yu Lu

1 Research Problem and Motivation

Large Language Model (LLM) self-training commonly bootstraps new supervision from model-generated reasoning traces, using correctness of the final answer as a proxy for trace quality. This proxy is unreliable for multi-step reasoning because a correct final answer can be produced with incorrect or redundant intermediate steps, which introduces noisy supervision and limits iterative improvement.

Outcome-only verification, such as an Outcome Reward Model (ORM) that scores only the final answer, does not provide step-level feedback needed to guide search and to refine intermediate reasoning. Process supervision via a Process Reward Model (PRM) provides step-wise evaluation, but conventional PRM training requires expensive human annotation of intermediate steps. The work targets an automatic procedure that (i) constructs fine-grained process supervision without human step labels, (ii) uses this signal to guide tree search for higher-quality reasoning traces, and (iii) iteratively improves both the reasoning policy and the process value model.

2 Related Work

LLM self-training and self-improvement

Representative self-training paradigms include ReSTEM, which refines generated chain-of-thought data using ground-truth final answers, and Self-Rewarding, which uses an LLM-based critic to judge outcomes. These approaches remain sensitive to intermediate-step errors because data selection primarily depends on final-answer correctness or coarse reward signals.

Search-based reasoning and verification

Tree-of-Thought (ToT) organizes reasoning as a search over partial solutions and typically relies on a value function to guide expansion. Monte Carlo Tree Search (MCTS) is a standard framework for balancing exploration and exploitation in a search tree, but performance depends on the quality of node values and the computational budget. Verification baselines include Self-Consistency (SC), ORM-based Best-of- N (BoN), and PRM-based selection schemes. MATH-SHEPHERD (MS) is included as a comparison verifier and reward-model training approach under controlled protocols.

3 Dataset Construction

The work constructs training data for a step-wise value model that serves as a PRM and supports tree search. The dataset construction is explicitly described for science, while math uses an alternative procedure

described in the paper.

Table 1: Datasets and benchmarks used in training and evaluation. Fields absent from the paper are marked as “Not specified in the paper.”

Name	Role	Description
SciInstruct	Source (science)	Lean science instruction dataset integrated for science value training. Scale: 11,554 questions, each paired with a correct step-by-step solution.
MATH (train)	Source (math)	Integrated for math value training; generation uses Mistral-7B: MetaMATH as a policy with a breadth-first search strategy; leaf answers are verified. Further details: Not specified in the paper.
MATH	Benchmark	Math reasoning benchmark used for evaluation of self-training and verification.
GSM8K	Benchmark	Grade-school math benchmark used for verifier comparison.
MATH500	Benchmark	A 500-problem subset used for verifier comparison.
SciBench	Benchmark	Science reasoning benchmark with multiple subjects (chemistry, physics, math subcategories) used to evaluate search and verification performance.
SciEval	Benchmark	Science evaluation benchmark used for search-based performance comparison. Additional details: Not specified in the paper.
GPQA Diamond	Benchmark	Hard QA benchmark used for self-training evaluation.
CEval-Hard	Benchmark	Hard evaluation split used for self-training evaluation.

Science value-model training data

Science questions are sampled from SciInstruct. For each question, partial solutions are extracted from step-by-step solutions to form training instances for step-wise evaluation. To inject negative (incorrect) step continuations, an incompetent LLM policy (ChatGLM2) is used to generate single-step continuations that are treated as incorrect. The resulting initial science value-training set is reported as **473.4k samples**.

Math value-model training data

For math, value training does not require human step labels and only demands a correct final answer. The MATH training set is integrated, a search tree is generated using Mistral-7B: MetaMATH under a breadth-first search strategy, and leaf-node answers are verified. Dataset size, filtering thresholds, and additional preprocessing are **Not specified in the paper**.

4 Query Protocol and Task Definitions

Input-output format and success criteria

Each instance provides a question and a ground-truth final answer. A reasoning trace is a sequence of steps that culminates in a final answer. Success is defined by exact match between the predicted and ground-truth final answers under the benchmark-specific evaluation scripts. The primary metric across reported benchmarks is **accuracy (%)**.

Verifier protocol and controlled comparisons

Verifier comparisons are conducted under a controlled setting where verification is based on **256 outputs** for each question. The protocol compares SC, ORM, combinations such as SC+ORM, MS, SC+MS, and SC combined with the proposed ReST-MCTS* value model.

Table 2: Verifier accuracy (%) on GSM8K and MATH500 under the controlled protocol where verification is based on 256 outputs. SC: Self-Consistency. MS: MATH-SHEPHERD.

Backbone	Dataset	SC	ORM	SC+ORM	MS	SC+MS	SC+ReST-MCTS* (Value)
Mistral-7B: MetaMATH	GSM8K	83.9	86.2	86.6	87.1	86.3	87.5
	MATH500	35.1	36.4	38.0	37.3	38.3	39.0

5 Modeling Approach

The method couples (i) an automatically trained process value model (PRM) and (ii) a process-reward-guided MCTS variant (MCTS*) to generate higher-quality reasoning traces for iterative self-training. A Policy Model (PM) generates candidate next-step continuations; a Value Model (VM) provides step-wise values to score partial solutions.

Core mechanism: weighted reward and quality value

The paper defines a step-wise weighted reward for the current step s_k and a recursive quality value for a partial solution prefix.

$$w_{s_k} = \frac{1 - v_{k-1}}{m_k + 1} (1 - 2r_{s_k}), \quad k = 1, 2, \dots \quad (1)$$

Purpose. Transform a step-wise score into a bounded weighted reward that updates the quality of a partial reasoning prefix and enables process-level credit assignment during search.

Symbols. w_{s_k} is the weighted reward of step k ; $v_{k-1} \in [0, 1]$ is the previous quality value; m_k is the remaining step budget term defined in the paper (with $m_k = K - k$ under the analysis in the paper); $r_{s_k} \in [0, 1]$ is the step-wise score output by the value model; k is the step index.

Intuition. The factor $(1 - 2r_{s_k})$ maps a score in $[0, 1]$ into a signed quantity in $[-1, 1]$, allowing penalization of low-quality steps. The multiplier $(1 - v_{k-1})/(m_k + 1)$ shrinks the update as the prefix quality approaches 1 and as the remaining step budget decreases.

Usage. w_{s_k} is computed for each expansion step during tree search and is used to update the prefix quality via Eq. (2), thereby guiding which partial solutions receive further exploration.

$$v_k = \max(v_{k-1} + w_{s_k}, 0) \quad (2)$$

Purpose. Define the recursive quality value of a partial solution prefix, which serves as the node quality signal used by the search algorithm and as a training target for the value model.

Symbols. $v_k \in [0, 1]$ is the quality value after k steps; v_{k-1} is the previous quality value; w_{s_k} is the weighted reward from Eq. (1).

Intuition. The update accumulates step-wise rewards while enforcing non-negativity via the max operator, preventing negative drift from dominating subsequent scoring.

Usage. During search, each node corresponding to a partial solution prefix is assigned v_k ; termination and selection rules compare this value to thresholds defined in the search procedure.

Search policy: UCB-based node selection in MCTS*

The appendix specifies an Upper Confidence Bound (UCB) criterion for selecting which child node to expand.

$$\text{UCB}(C) = v_C + \epsilon \sqrt{\frac{\ln n_{\text{parent}}}{n_C}} \quad (3)$$

Purpose. Balance exploitation of high-quality nodes with exploration of under-visited nodes during tree search.

Symbols. C is a candidate child node; v_C is the node quality value; n_{parent} is the visit count of the parent node; n_C is the visit count of node C ; ϵ is an exploration constant.

Intuition. Nodes with higher quality value v_C are preferred, while the exploration bonus increases when n_C is small, ensuring that potentially good but under-sampled branches are explored.

Usage. In each MCTS* iteration, the selection step chooses the child with maximal UCB score and proceeds until a leaf node is reached for expansion or termination.

End-to-end procedure and a minimal example

A question is placed at the root. The policy samples candidate next steps to create child nodes. For each partial prefix, the value model evaluates step quality to compute r_{s_k} , then updates v_k via Eqs. (1) and (2). MCTS* repeats selection, expansion, evaluation, and backup to identify a high-quality reasoning trace. The final answer extracted from the trace is checked against the ground truth; accepted traces are used to update the policy and the value model in subsequent self-training iterations.

Table 3: Key modeling components and reported hyperparameters. Missing items are “Not specified in the paper.”

Item	Setting
Policy model (PM)	LLaMA-3-8B-Instruct; Mistral-7B; MetaMATH; SciGLM-6B (used in the self-training comparison).
Value model (VM) initialization (science)	Fine-grained value training set built from SciInstruct; total size 473.4k samples.
Value model (VM) initialization (math)	Uses Mistral-7B; MetaMATH as a policy and breadth-first search to generate a search tree; leaf answers are verified. Additional details: Not specified in the paper.
MCTS* termination threshold	A threshold l is set to 0.9 in experiments for determining whether a selected node is acceptable as a final solution in the described procedure.
Search branching factor	Not specified in the paper.
Number of MCTS* iterations	Not specified in the paper.
Sampling settings (for reward-model comparison on SciBench)	Number of samples for each step is set to 3.
Training optimizer, learning rate, batch size, steps	Not specified in the paper.
Hardware and training cost	Not specified in the paper.

6 Empirical Results

Iterative self-training on policy and value models

Primary self-training results compare ReST-MCTS* with ReSTEM and Self-Rewarding across multiple iterations, evaluated on MATH, GPQA Diamond, and CEval-Hard. Accuracy values are reported in Table 4.

Table 4: Primary results (accuracy %) for training both policy and value models over multiple self-training iterations. Evaluation is zero-shot; the few-shot baseline serves as a comparison.

Backbone	Method / Iteration	MATH	GPQA Diamond	CEval-Hard	Ave.
LLaMA-3-8B-Instruct	0th (zero-shot)	20.76	27.27	26.32	24.78
	0th (few-shot)	30.00	31.31	25.66	28.99
	ReSTEM (1st)	30.84	26.77	21.05	17.22
	Self-Rewarding (1st)	30.34	26.26	25.66	27.42
	ReST-MCTS* (1st)	31.42	24.24	26.97	27.55
	ReSTEM (2nd)	33.52	25.25	21.71	26.83
	Self-Rewarding (2nd)	33.89	26.26	23.03	27.73
	ReST-MCTS* (2nd)	34.28	27.78	25.00	29.02
Mistral-7B: MetaMATH	0th (zero-shot)	29.34	27.78	9.87	22.33
	0th (few-shot)	28.28	29.29	9.21	22.26
	ReSTEM (1st)	23.84	26.26	20.39	23.50
	Self-Rewarding (1st)	25.70	27.78	19.74	24.40
	ReST-MCTS* (1st)	31.06	26.26	17.11	24.81
	ReSTEM (2nd)	23.86	26.26	22.37	24.16
	Self-Rewarding (2nd)	23.90	26.77	25.00	25.22
	ReST-MCTS* (2nd)	24.40	28.79	26.32	26.50
SciGLM-6B	0th (zero-shot)	25.18	23.74	51.97	33.63
	ReSTEM (1st)	22.72	24.75	51.32	32.93
	Self-Rewarding (1st)	22.50	26.26	47.37	32.04
	ReST-MCTS* (1st)	24.86	25.25	51.32	33.81
	ReSTEM (2nd)	25.86	25.25	48.68	33.27
	Self-Rewarding (2nd)	23.86	28.79	48.03	33.56
	ReST-MCTS* (2nd)	23.90	31.82	51.97	35.90

Table 4 indicates that ReST-MCTS* achieves strong average accuracy after two iterations for LLaMA-3-8B-Instruct (Ave. 29.02) and SciGLM-6B (Ave. 35.90). For Mistral-7B: MetaMATH, ReST-MCTS* shows the highest reported average after two iterations (Ave. 26.50), driven by large gains on CEval-Hard (26.32). A trade-off is visible in the first iteration for Mistral-7B: MetaMATH, where MATH increases to 31.06 while CEval-Hard is 17.11, indicating that improvements are not uniformly distributed across benchmarks under the reported training setup.

SciBench: benchmark performance across models and subjects

Overall SciBench performance compares Chain-of-Thought (CoT), ToT, and ReST-MCTS* across GLM4, GPT-3.5-turbo, and LLaMA2-13B-Chat. The paper reports averages over 10 subjects and highlights that larger models benefit more from the search-based procedure.

Table 5: SciBench overall accuracy (%) averaged across subjects for three representative backbones.

Backbone	CoT (Ave.)	ToT (Ave.)	ReST-MCTS* (Ave.)
GLM4	12.68	15.82	16.77
GPT-3.5-turbo	6.92	8.44	10.06
LLaMA2-13B-Chat	2.23	2.37	2.90

The subject-level table in the paper shows that improvements are concentrated in specific categories.

For GLM4, chemistry subcategory “chemmc” increases from 23.07 (CoT) to 28.20 (ReST-MCTS*), and chemistry subcategory “quan” increases from 8.82 (CoT) to 14.70 (ReST-MCTS*). For GLM4 math subcategory “count”, accuracy increases from 12.00 (CoT) to 26.66 (ReST-MCTS*). These values indicate that the process-reward-guided search is particularly beneficial on subdomains where single-sample chain-of-thought generation struggles to consistently produce correct intermediate reasoning.

Efficiency and cost

The paper reports average running time and accuracy on MATH for several algorithms under a basic experiment setting.

Table 6: Average running time (seconds) and MATH accuracy (%) under the paper’s basic experiment settings.

Metric	CoT+SC	ORM+BoN	PRM+BoN	MCTS*
Running time (s)	41	43	73	108
MATH accuracy (%)	37.0	33.5	34.5	41.5

The reported results show that MCTS* attains higher MATH accuracy (41.5%) than CoT+SC (37.0%) and BoN variants, while incurring higher average running time (108 seconds per question). This establishes a measurable accuracy-cost trade-off.

Ablation-style comparison of reward and value guidance

To isolate the effect of different reward or value models under the same search policy, the paper evaluates ORM, PRM, Self-Rewarding, and the ReST-MCTS* value model on SciBench questions while using MCTS* as the search procedure.

Table 7: Accuracy (%) of different reward/value models on selected SciBench questions under MCTS* as the search policy.

Backbone	ORM	PRM	Self-Rewarding	ReST-MCTS* (Value)
GLM4	20.2	22.0	20.2	22.9
GPT-3.5-turbo	12.8	17.4	13.7	20.2

The comparisons indicate that PRM-style guidance improves over ORM under the same MCTS* search policy, and the proposed ReST-MCTS* value model further increases accuracy for GPT-3.5-turbo (20.2) relative to PRM (17.4) and Self-Rewarding (13.7), supporting the claim that fine-grained process evaluation is a key driver of search effectiveness.

7 Summary

The work introduces a self-training framework that integrates process-reward-guided tree search with automatic construction of step-wise supervision for a value model. The core contribution is a value definition that updates a prefix quality value through a bounded weighted reward (Eqs. (1) and (2)) and uses this signal within a UCB-guided MCTS* policy (Eq. (3)). Empirical results show consistent gains in iterative self-training for multiple backbones (Table 4), stronger verifier performance under a controlled 256-output protocol (Table 2), improved SciBench averages for larger models (Table 3), and measurable accuracy-cost

trade-offs relative to simpler baselines (Table 4), with additional evidence that PRM-style guidance improves search outcomes (Table 5).

Limitations reported in the paper include: (i) generalization to tasks without oracle labels, since the approach relies on supervised datasets with ground-truth answers for initial filtering; (ii) insufficient scale and diversity of the proposed value model backbone, with a larger backbone required for stronger PRM training; (iii) limited generality of the initial PRM training data generation, which is produced by SciGLM and thus focused on mathematical and scientific reasoning; and (iv) the need for improved self-training data filtering techniques and stronger value model backbones to further increase gains.